

MANYFARM / HCR-3A

선별·운반 작업

Rodi 프로그래밍 실습서

노트북 준비 · 웹캠 구역 설정 · ROS 2 더미 관제
Modbus 상태 교환 · PICK / PLACE / 복귀 프로그램

Rodi 2.009.000 / Build 008_G2
CLINK v2.009.000.008 · 2026-10-01

따라가는 순서: 노트북 시험 → 펜던트 작성 → 실물 검증
그리퍼 제어 및 이 빌드의 세부 UI는 현장 보완 대상

HCR-3A 노트북 작업 순서

2026-10-01. Ubuntu 24.04 + ROS 2 Jazzy 기준. HCR-3A 선별 작업이며 SO-ARM101 수확/OMX 시연 수집과 별개다.

1. 최신 코드 받기

처음 받는 노트북:

```
git clone https://github.com/Robot3FinalProject/sorting.git
cd sorting
```

이미 받은 노트북은 해당 sorting 폴더에서:

```
git status --short
git pull --ff-only
```

로컬 수정 때문에 pull이 실패하면 덮어쓰지 말고 수정 내용을 먼저 보존한다. 아래 모든 명령은 sorting 저장소 루트 기준이다. 기존 OMX의 scripts/setup.sh는 HCR ROS 설치 스크립트가 아니다.

2. ROS 환경과 빌드

Ubuntu 24.04에서 ROS 2 Jazzy가 설치되어 있어야 한다. 설치가 없다면 공식 안내 <https://docs.ros.org/en/jazzy/Installation/Ubuntu-Install-Debs.html> 를 먼저 따른다. Ubuntu 22.04에 Jazzy 명령을 그대로 적용하지 않는다.

```
source /opt/ros/jazzy/setup.bash
sudo apt update
sudo apt install -y python3-colcon-common-extensions python3-opencv python3-numpy ros-jazzy-rclpy ros-jazzy-rosidl-default-
generators ros-jazzy-std-srvs
bash src/sorting_station/hcr/build_sorter.sh
```

빌드 결과는 outputs/sorting_station/ros2에 생성된다. 다른 PC의 venv/build/install을 복사하지 않는다. 웹캠은 시스템 Python/OpenCV를 사용한다.

3. 먼저 로봇 없이 시험

Rodi 작성 가이드 8절의 터미널 4개를 순서대로 실행한다: 웹캠 → localhost 실행기 → mock_hcr → 더미 관제. a/b/c/d로 사각형을 지정하고 s로 저장한다. 더미 관제에서는 p로 ready 확인 후 s+Enter로 시작한다. 영상 창의 s는 구역 저장, 관제 터미널의 s는 작업 시작이므로 구분한다.

A 빨간을 보여 명령11과 BUSY/완료 횟수를 확인한다. mock은 물체를 실제로 치우지 않으므로 계속 보이면 반복한다. 물체를 치우면 3초 미검출 후 SUCCESS다. c는 새 작업을 막고 진행 중 1회가 끝난 뒤 취소한다.

4. Rodi 작성

함께 제공하는 가이드의 2~7절을 따라 신호 → 변수 → 티칭 위치 → PICK_A/B/C/D → PLACE_AND_RETURN → MAIN을 만든다. 표기된 트리는 설명용이며 붙여넣기 스크립트가 아니다. 그리퍼는 미구현으로 남겨 둔다. 주기 쓰기/실제 이동을 검증하기 전 실물 동작을 시작하지 않는다.

5. 실물 연결로 전환

```
ip -br addr
# 아래 NIC_NAME은 랜선이 연결된 실제 유선 인터페이스 이름으로 바꾼다.
sudo ip addr add 192.168.3.2/24 dev NIC_NAME
ping -I 192.168.3.2 -c 4 192.168.3.100
```

이미 .2 주소가 있으면 추가 명령을 반복하지 않는다. 이는 임시 IP이므로 재부팅 후 확인한다. 로컬 mock 실행기와 mock_hcr를 종료하고 실제 실행기를 시작한다:

```
sudo -E bash src/sorting_station/hcr/run_sorter.sh
```

웹캠과 더미 관제는 그대로 사용한다. 이때 관제 s는 실제 HCR 작업 요청이다. 실제 운반 결과는 그리퍼와 놓기 성공 확인까지 구현한 뒤 확인한다.

6. 완료 확인과 기록

관제 Result의 red_count+yellow_count가 운반 횟수다. 실패/취소에도 이미 완료한 횟수는 유지된다. 빈 영상은 레인 상류가 비었다는 센서 보장이 아니다. 카메라 정지/미설정은 빈 영상 성공으로 처리하지 않는다.

PC 오류는 물리 비상정지가 아니다. 실물 상태를 확인하고 복구한 뒤 reset_fault를 사용한다. 저널을 삭제해 오류를 숨기거나 불확실한 작업을 자동 재시도하지 않는다.

소프트웨어 검증: 런타임 단위 테스트 17개, 카메라 테스트 6개, ROS/Modbus 통합 시험. Rodi 실제 동작과 그리퍼는 별도 현장 검증 대상이다.

HCR-3A Rodi 선별 프로그램 작성 순서

작성: 2026-10-01. 대상은 HCR-3A 선별 로봇팔이며 SO-ARM101이 아니다.

1. 버전과 이 문서의 사용법

첨부 사진에서 확인한 버전:

항목	버전
Rodi	2.009.000 / Build 008_G2
제어기	CLINK v2.009.000.008
안전 모듈	1.002.007
툴 I/O	2.07
I/O 모듈	1.000.001
관절 1~6 드라이버	0.28

이 문서는 **펜던트에서 수동으로 만드는 프로그램의 설계도**다. 가져오기 가능한 Rodi 프로젝트 파일이나 그대로 붙여넣는 스크립트가 아니다. 제조사 [자료실](#)에 HCR 2nd Generation 사용자 매뉴얼 및 Rodi Script Programming Guide가 있다. 다만 해당 빌드의 상세 매뉴얼 본문과 실제 편집 화면을 대조하지 못했으므로 버튼 위치·조건식 문법·병렬 실행 설정은 확정하지 않는다.

아래 SET, IF, LOOP, WAIT, CALL, MOVE L/J는 기능을 설명하는 표기다. 화면의 변수 설정, 조건, 반복, 대기, 서브프로그램 실행, L/J 이동 기능으로 옮긴다. 이름은 직접 만들어도 되는 프로그램/변수 이름이다. AND, != 등은 설명용 논리식이며 화면의 조건 편집기로 작성한다.

작성 순서: 통신 신호 등록 → 이동 없는 통신 시험 → 위치 티칭 → PICK 4개 → 운반 공통 루틴 → 메인 연결. 아직 그리퍼 정보가 없으므로 실제 집기/놓기 부분은 미완성으로 남긴다.

2. Modbus 신호부터 등록

HCR은 클라이언트, 노트북은 서버다. 기존 PC 디바이스 IP 192.168.3.2, Slave ID 1을 사용한다. 로봇 IP는 192.168.3.100. 기존 LS 디바이스는 변경하지 않는다. 신호 수량은 각각 1, 업데이트 주기는 우선 10 Hz로 설정한다. 주소는 아래 정수 그대로이며 40001을 더하지 않는다.

Signal Address	타입	이름	의미
0	Register Input	PC_COMMAND	구역/색상 명령
1	Register Input	PC_JOB_ID	이번 운반 번호
2	Register Input	PC_REQUEST	1이면 새 작업 요청
3	Register Input	PC_BATCH_ACTIVE	관제 배치 활성화
4	Register Input	PC_HEARTBEAT	계속 바뀌는 PC 생존 값
100	Register Output	HCR_STATE	0미준비, 1READY, 2BUSY, 3DONE, 4ERROR
101	Register Output	HCR_ACK_JOB	수락한 운반 번호
102	Register Output	HCR_DONE_JOB	완료한 운반 번호
103	Register Output	HCR_ERROR	0정상, 비영 오류

기존 PC_TEST가 주소0이면 같은 입력이다. 중복 신호 대신 이름을 정리한다. 아래 '출력 설정'은 해당 Modbus Register Output 값을 쓰는 기능을 뜻하며 물리 Digital Output을 뜻하지 않는다.

구역	빨강 → 왼쪽 박스	노랑 → 오른쪽 박스
A	11	12
B	21	22
C	31	32
D	41	42

먼저 해결해야 할 주기 쓰기

PC는 주소100에 실제로 들어오는 쓰기 요청을 heartbeat로 사용한다. 읽기만 반복하거나 값이 바뀔 때만 쓰면 3초 후 통신 오류다. 이동 중에도 STATE를 최소 1Hz, 권장 10Hz로 반복 전송해야 한다.

다음 중 실제 장비에서 확인되는 방법을 선택한다.

1. Register Output의 주기 갱신이 값이 같아도 실제 쓰기 패킷을 전송한다면 이를 사용한다.
2. 그렇지 않으면 병렬 스레드/서브프로그램에서 주기 쓰기를 구현한다. **이 빌드에서 이동과 병렬 쓰기가 가능한지 먼저 확인해야 한다.** 불가능하면 현재 프로토콜을 그대로 실물에 적용하지 말고 출력 갱신 방식에 맞게 PC와 함께 수정한다.

통신 상태 송신은 한 루틴이 담당하는 편이 좋다. 로컬 상태 묶음을 복사하여 ACK → DONE → ERROR → STATE 순으로 쓰고 0.1초 대기한다. STATE는 마지막 commit이다. 상태 전환과 주기 송신이 서로 끼어들어 새 STATE와 이전 ACK가 섞이지 않도록 동기화한다. 100~103을 하나의 FC16으로 원자 쓰기할 수 있으면 그 방법이 가장 단순하다. **GUI에서 값을 설정한 순서가 네트워크 쓰기 순서를 보장하는지는 별도로 검증한다.**

READY를 10초 이상 유지해 PC의 ready가 계속 true인지 확인하고, 이동 없는 5초 대기 BUSY 시험에서도 통신 오류가 없는지 확인한다. 이 단계가 통과하기 전 모션을 연결하지 않는다.

3. 로컬 변수 만들기

변수	초기값	용도
active_job	0	현재 작업 번호 고정 보관
active_command	0	현재 명령 고정 보관
last_job	0	같은 작업 중복 실행 방지
is_red	0	1빨강 / 0노랑
state_out	0	송신할 상태
ack_out	0	송신할 ACK
done_out	0	송신할 DONE
error_out	0	송신할 오류
wait_count	0	통신 대기 제한용

병렬 송신을 사용한다면 공유 가능한 변수 범위로 만든다. active_command는 BUSY가 된 후 절대 PC_COMMAND로 덮어쓰지 않는다.

4. 위치를 먼저 티칭하여 저장

엑셀 수치는 참고값이다. **좌표만 입력해서 완성할 수 없다.** TCP, 기준 좌표계, 자세 Rx/Ry/Rz, 실제 접근/집기 높이가 빠져 있다. 실제 장비에서 전체 자세를 저장하고 같은 TCP/좌표계를 사용한다. 표의 mm/deg는 자료 해석이며 펜던트 단위도 확인한다.

저장 이름	내용	엑셀 참고 XYZ (mm)
P_HOME	선별장 초기 위치	176.26, 227.42, 168.73
P_A_UP	A 접근/상승 완료	-172.08, 227.42, 168.73
P_B_UP	B 접근/상승 완료	-46.67, 227.42, 168.73
P_C_UP	C 접근/상승 완료	81.93, 227.42, 168.73
P_D_UP	D 접근/상승 완료	210.87, 227.42, 168.73
P_A_PICK ~ P_D_PICK	각 레인 파지 위치	아래 이동 높이 미정, 각각 티칭
P_RIGHT	오른쪽 박스 놓기 위치	-60.83, -400.2, 672.04 참고
P_LEFT	왼쪽 박스 놓기 위치	전체 TCP 자세 직접 티칭

J 이동용으로는 각 위치의 **6개 관절 전체**를 저장한다. 각 행은 이전 행에서 지정 관절만 바꾼 사용자 의도를 풀어 쓴 참고값이다.

이름	J1	J2	J3	J4	J5	J6
Q_HOME	-275.17	-42.26	-99.02	-121.65	-270.97	-1.83
Q_3	-275.17	-42.26	-47.81	-121.65	-270.97	-1.83
Q_34	-275.17	-42.26	-47.81	-85.46	-270.97	-1.83
Q_345	-275.17	-42.26	-47.81	-85.46	-124.92	-1.83

P_RIGHT와 Q_345는 동일한 실제 자세에서 저장한다. -275도 등을 임의로 +85도로 바꾸지 않는다. 같은 TCP 위치로 L 복귀해도 관절 구성이 같다는 보장은 없으므로 관절값도 확인한다. J 이동 중 TCP 경로는 직선이 아니다. 처음에는 블렌딩 없이 낮은 시험 속도로 각 구간을 확인한다. 임의의 현재 위치에서 P_HOME으로 바로 이동시키는 시작 루틴은 만들지 않는다.

5. PICK 서브프로그램 4개

PICK_A를 다음 순서로 만들고 B/C/D용으로 복사하여 대상 위치만 바꾼다.

순번	추가할 기능	설정/목적
1	L 이동	P_A_UP
2	L 이동	P_A_PICK
3	그리퍼 닫기 자리	현재 미구현. 실제 운반 프로그램에서는 여기서 진행을 막는다.
4	파지 성공 확인 자리	센서/완료 조건 추후 정의. 단순 시간 경과를 성공으로 간주하지 않는다.
5	L 이동	P_A_UP
6	L 이동	P_HOME

PICK_B는 P_B_UP/P_B_PICK, C와 D도 동일하다. 호출 전 P_HOME에 있어야 한다. 그리퍼 없는 경로 시험은 별도 프로그램 복사본에서 해당 자리를 주석/대기로 바꾸고 빈 작업 공간에서 수행한다. 그 복사본이 보내는 DONE은 시험 횟수일 뿐 실제 운반 성공이 아니다.

6. PLACE_AND_RETURN 공통 서브프로그램

PICK 종료 후 호출한다.

순번	기능	대상/조건
1	J 이동	Q_3
2	J 이동	Q_34
3	J 이동	Q_345: 오른쪽 박스 도착
4	IF	is_red = 1이면 L 이동 P_LEFT
5	그리퍼 열기 자리	미구현, 실제 운반 진행 차단
6	놓기 성공 확인 자리	추후 보충
7	IF	is_red = 1이면 L 이동 P_RIGHT
8	J 이동	Q_34: J5 복귀
9	J 이동	Q_3: J4 복귀
10	J 이동	Q_HOME: J3 복귀

노랑은 4/7의 좌우 이동이 없다. 빨강은 오른쪽 위치를 거쳐 왼쪽에서 놓고 다시 오른쪽으로 온다. 박스 테두리와 적재물 위를 지나는 L 경로를 확인하고 필요하면 안전한 높이의 중간 티칭점을 추가한다. 마지막 복귀까지 완료되어야 DONE이다.

7. 메인 프로그램 트리

아래를 위에서부터 작성한다. 조건 대기는 지원되는 WAIT 조건 기능 또는 LOOP + IF + 시간 대기 0.1초로 만든다. 무한 조건 대기 대신 10초 제한(0.1초 100회)을 둔다. 오류 발생 시 정상 경로를 빠져나와 오류 유지 루틴에 머무른다.

MAIN

초기화: state_out=0, error_out=0, ack_out=0, done_out=0
 상태 주기 송신 시작 (2절 검증 필수)
 PC 통신 정상 / HEARTBEAT 값 변화 확인
 PC_REQUEST=0 AND PC_COMMAND=0 확인 (재시작 시 이전 요청 자동 재실행 금지)
 운영자가 초기 위치와 작업 공간을 확인한 시작 조건 확인
 state_out=1 [READY]

반복

대기: PC_BATCH_ACTIVE=1 AND PC_REQUEST=1
 active_job = PC_JOB_ID
 active_command = PC_COMMAND
 다시 확인: REQUEST=1 / BATCH_ACTIVE=1 / JOB_ID가 active_job과 동일
 유효성 확인: active_job>0, active_job!=last_job
 유효성 확인: active_command가 11,12,21,22,31,32,41,42 중 하나
 불일치/불법 요청 → 오류 루틴, 모션 금지

ack_out=active_job
 state_out=2 [BUSY: ACK와 일관된 묶음으로 송신]
 대기: PC_REQUEST=0 [10초 제한, 통신 정상 유지]
 모션 직전 PC_COMMAND=active_command 및 PC_BATCH_ACTIVE=1 재확인

명령 분기 (아래 표의 8개 가지)
 PLACE_AND_RETURN 실행
 정상 놓기 + HOME 복귀 성공 확인

last_job=active_job
 done_out=active_job
 state_out=3 [DONE: DONE_JOB과 일관된 묶음으로 송신]
 대기: PC_COMMAND=0 AND PC_REQUEST=0 [10초 제한]
 state_out=1 [READY]
 반복 처음으로

조건	분기 안에서 순서대로 수행
active_command = 11	is_red=1 → PICK_A
active_command = 12	is_red=0 → PICK_A
active_command = 21	is_red=1 → PICK_B
active_command = 22	is_red=0 → PICK_B
active_command = 31	is_red=1 → PICK_C
active_command = 32	is_red=0 → PICK_C
active_command = 41	is_red=1 → PICK_D
active_command = 42	is_red=0 → PICK_D
그 밖의 값	오류 유지, PICK/PLACE 실행 금지

SWITCH를 사용하거나 상호 배타적인 IF 분기 8개로 만든다. PLACE_AND_RETURN은 선택된 PICK 뒤에 한 번만 호출한다. READY로 돌아온 후 카메라에 A가 또 있으면 새 job으로 A를 다시 실행하는 것이 정상이다.

오류 경로

로컬 오류 코드 제안: 1잘못된명령, 2수락응답시간초과, 3PC통신이상, 4모션/파지/놓기실패, 5완료응답시간초과. 이는 새로 제안한 값이며 장비 고유 오류번호가 아니다. PC는 비영 값을 실패로 처리한다.

오류 시 error_out=비영 → state_out=4로 일관되게 송신하고 신규 작업을 받지 않는다. 실패 후 DONE이나 READY를 자동으로 보내지 않는다. 실제 모션 중 통신 단절의 정지/물체 유지 방식은 로봇의 검증된 정지 기능과 그리퍼 특성에 맞춰 따로 설정해야 한다. PC 오류나 ROS 취소는 비상정지가 아니다.

병렬 감시에서는 PC_HEARTBEAT가 3초 이상 바뀌지 않거나 Modbus 연결이 끊기면 새 동작을 차단한다. 이동 중 감시/정지와 상태 송신의 실제 구현은 펜던트에서 검증할 부분이다. ACK/완료 대기에서 시간 초과했다고 다음 이동으로 넘어가면 안 된다.

8. 관제 없이 시험: 더미 관제 사용

저장소 루트에서 한 번 빌드:

```
bash src/sorting_station/hcr/build_sorter.sh
```

먼저 로봇 없이 전체 통신 시험

각각 다른 터미널에서 실행한다.

```
# 1: 웹캠, a/b/c/d를 누르고 사각형 드래그, s로 저장
bash src/sorting_station/hcr/run_webcam_preview.sh --camera 0
```

```
# 2: 로컬 PC 실행기, 실제 로봇 IP에 연결하지 않음
bash src/sorting_station/hcr/run_sorter.sh --ros-args \
  -p modbus_host:=127.0.0.1 -p modbus_port:=1502 -p modbus_peer:=127.0.0.1 \
  -p journal:=outputs/sorting_station/hcr_runtime/mock_journal.sqlite3
```

```
# 3: 로봇 응답 흉내 (관제 흉내와 다른 프로그램)
source /opt/ros/jazzy/setup.bash
source outputs/sorting_station/ros2/install/setup.bash
ros2 run hcr_sorting mock_hcr --host 127.0.0.1 --port 1502 --cycle-seconds 2
```

```
# 4: 더미 관제
bash src/sorting_station/hcr/run_dummy_control.sh
```

더미 관제 터미널에서 명령을 입력하고 Enter를 누른다.

입력	기능
p	현재 상태, ready, 상태 메시지 나이 확인
s	새 task/batch ID로 Goal 1개 전송. ready=false면 전송하지 않음
c	취소 요청. 진행 중 1회는 복귀까지 기다린 후 CANCELED
q	활성 작업 없을 때 종료

실행만으로 Goal을 보내지 않는다. 시작 시 실제 하역 완료와 운송로봇 이격은 사용자가 확인한다. 더미 관제는 이 센서를 흉내 내거나 자동 보증하지 않는다. 결과에는 빨강/노랑/합계와 SUCCESS/FAILED/CANCELED가 표시된다. 취소 접수 메시지는 최종 완료가 아니다.

mock_hcr는 실제 과일을 옮기지 않으므로 계속 같은 물체가 보이면 계속 횡수가 증가한다. 물체를 화면에서 치우면 연속 3초 미검출 후 SUCCESS. Ctrl+C/터미널 종료는 실행기를 멈추거나 실제 로봇을 정지시키지 않는다. 가능하면 c → 최종 Result → q 순으로 종료한다.

실제 HCR 연결 시험

로컬 시험의 2/3을 종료한다. 실제 통신용 실행기는 아래로 바꾼다. 웹캠과 더미 관제는 그대로 사용한다. **mock_hcr는 실행하지 않는다.**

```
sudo -E bash src/sorting_station/hcr/run_sorter.sh
```

PC에 192.168.3.2가 설정되어 있어야 한다. 예전 Modbus 테스트 서버와 동시에 502를 점유할 수 없다. 모든 ROS 터미널의 ROS_DOMAIN_ID를 같게 한다. Rodi 통신 루틴이 READY를 보내고 카메라 4구역이 준비되면 p에서 ready=True가 된다. 이 상태의 s는 실제 프로그램을 시작시키는 요청이다.

권장 검증 순서:

1. 이동 없이 READY 10초 유지.
2. 이동 대신 5초 대기로 BUSY → DONE → READY 한 회 검증. 시험 수량으로만 해석.
3. 빈 공간에서 위치별 단일 이동과 귀환 검증. 아직 실제 운반 성공으로 집계하지 않음.
4. 그리퍼 닫기/열기 및 성공 판정을 채운 뒤 실제 1개 작업 검증.
5. A 연속 2개, A/B 동시 입력 우선순위, 빨강/노랑 분리, 빈 영상 종료, 운반 중 취소 검증.

실물에서 아직 미검증인 항목: 이 빌드의 Modbus 반복 쓰기/순서, 이동 중 병렬 감시, 각 이동 경로, 그리퍼 및 정상 놓기 판정. 이 문서만으로 이 항목들이 검증된 것으로 간주하지 않는다.

부록: 더미 관제 Python 전체 코드

설치된 코드: src/sorting_station/hcr_sorting/hcr_sorting/dummy_control.py. 아래는 읽기용이며 파일은 저장소에 포함되어 있다.

```
"""Interactive central-control substitute. Sending a goal can move real hardware."""
import argparse
from datetime import datetime, timezone
import select
import sys
import time
import uuid

import rclpy
from rclpy.action import ActionClient
from rclpy.node import Node
from smartfarm_interfaces.action import SortBatch
from smartfarm_interfaces.msg import SorterStatus

class DummyControl(Node):
    def __init__(self, robot_id='sorter_01'):
        super().__init__('dummy_sorter_control')
        self.robot_id = robot_id
        self.status = None
        self.status_at = 0.0
        self.busy = False
        self.goal = None
        self.cancel_requested = False
        self.cancel_sent = False
        self.last_result = None
        self.last_feedback = None
        self.last_error = ''
        self.client = ActionClient(self, SortBatch, f'/{robot_id}/sort_batch')
        self.subscription = self.create_subscription(
            SorterStatus, f'/{robot_id}/status', self.on_status, 10)

    def on_status(self, message):
        if message.robot_id == self.robot_id:
            self.status, self.status_at = message, time.monotonic()

    def ready(self):
        return bool(not self.busy and self.status and self.status.ready
                    and time.monotonic() - self.status_at < 3.0
                    and self.client.server_is_ready())

    def report(self):
        if not self.status:
            return '아직 status를 받지 못했습니다. 실행기/ROS_DOMAIN_ID를 확인하세요.'
        s = self.status
        return (f'state={s.state} ready={s.ready} status_age={time.monotonic()-self.status_at:.1f}s '
                f'task={s.current_task_id} error={s.error_code} local_active={self.busy}')

    def start(self, task_id=None, batch_id=None):
```

```

if not self.ready():
    self.get_logger().warning('시작 불가: ' + self.report())
    return False
suffix = uuid.uuid4().hex
goal = SortBatch.Goal(robot_id=self.robot_id,
                      task_id=task_id or f'DUMMY_{suffix}',
                      batch_id=batch_id or f'BATCH_{suffix}',
                      timestamp=datetime.now(timezone.utc).isoformat())

self.busy = True
self.goal = None
self.cancel_requested = self.cancel_sent = False
self.last_result = self.last_feedback = None
self.last_error = ''
self.get_logger().info(f'Goal 전송 task={goal.task_id} batch={goal.batch_id}')
self.client.send_goal_async(goal, feedback_callback=self.on_feedback).add_done_callback(self.on_goal)
return True

def on_goal(self, future):
    try:
        self.goal = future.result()
        if not self.goal.accepted:
            self.busy = False
            self.last_error = 'GOAL_REJECTED'
            self.get_logger().warning('Goal 거절됨. 자동 재전송하지 않습니다.')
            return
        self.get_logger().info('Goal 수락됨')
        self.goal.get_result_async().add_done_callback(self.on_result)
        if self.cancel_requested:
            self.cancel()
    except Exception as exc:
        self.unknown(exc)

def on_feedback(self, message):
    f = message.feedback
    key = (f.phase, f.red_count, f.yellow_count)
    if key != self.last_feedback:
        self.get_logger().info(f'{f.phase}: 빨강={f.red_count} 노랑={f.yellow_count} 합계={f.red_count+f.yellow_count}')
    self.last_feedback = key

def on_result(self, future):
    try:
        result = future.result().result
        self.last_result = result
        self.busy = False
        self.get_logger().info(
            f'{result.result_code}: 빨강={result.red_count} 노랑={result.yellow_count} '
            f'총={result.red_count+result.yellow_count} error={result.error_code} {result.error_message}')
    except Exception as exc:
        self.unknown(exc)

def unknown(self, exc):
    # An uncertain request must never be silently retried as a fresh task.
    self.last_error = str(exc)
    self.get_logger().error(f'요청 결과 불확실: {exc}. 새 작업을 차단합니다. 실제 상태를 확인하세요.')

def cancel(self):
    if not self.busy:

```

```

        return False
    self.cancel_requested = True
    if self.goal is not None and self.goal.accepted and not self.cancel_sent:
        self.cancel_sent = True
        self.goal.cancel_goal_async().add_done_callback(self.on_cancel)
    return True

def on_cancel(self, future):
    try:
        response = future.result()
        if response.goals_canceling:
            self.get_logger().info('취소 수락: 진행 중 1회 완료 및 복귀 후 최종 Result를 기다립니다.')
        else:
            self.cancel_sent = False
            self.get_logger().warning('취소 거절 또는 이미 종료됨. Result를 기다립니다.')
    except Exception as exc:
        self.unknown(exc)

def main(args=None):
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument('--robot-id', default='sorter_01')
    opts, ros_args = parser.parse_known_args(args)
    rclpy.init(args=ros_args)
    node = DummyControl(opts.robot_id)
    print('더미 관제: s=시작(하역 완료·운송로봇 이격 확인 후), p=상태, c=취소, q=종료', flush=True)
    print('실제 실행기에 연결하면 s는 실제 로봇 작업을 요청합니다. 취소는 비상정지가 아닙니다.', flush=True)
    try:
        while rclpy.ok():
            rclpy.spin_once(node, timeout_sec=0.1)
            if not select.select([sys.stdin], [], [], 0)[0]:
                continue
            line = sys.stdin.readline()
            if not line:
                node.get_logger().warning('입력 종료. 활성 작업은 자동 정지되지 않습니다.')
                break
            command = line.strip().lower()
            if command == 's': node.start()
            elif command == 'p': print(node.report(), flush=True)
            elif command == 'c': node.cancel()
            elif command == 'q':
                if node.busy:
                    print('진행 중입니다. c로 취소하고 최종 Result 후 q를 입력하세요.', flush=True)
                else: break
    except KeyboardInterrupt:
        print('관제 종료. 실행 중인 로봇 작업은 자동 정지되지 않습니다.', flush=True)
    finally:
        node.client.destroy()
        node.destroy_node()
        if rclpy.ok(): rclpy.shutdown()

if __name__ == '__main__':
    main()

```